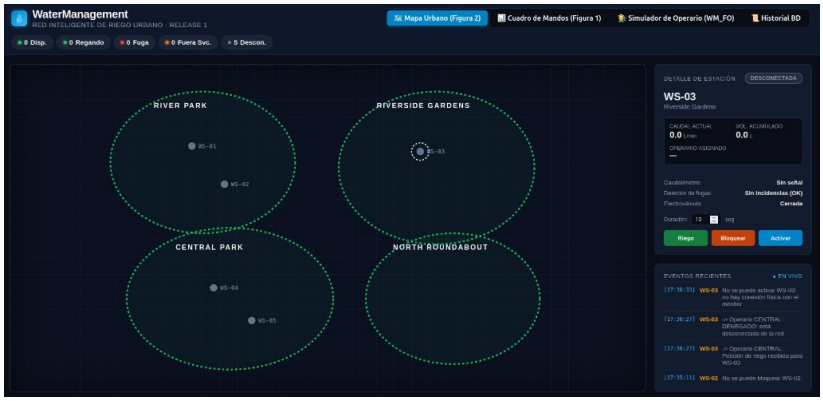
UNIVERSIDAD DE ALICANTE

Escuela Politécnica Superior · Grado en Ingeniería Informática

MEMORIA DE PRÁCTICAS: WATERMANAGEMENT NETWORK

Sistemas Distribuidos (Release 1)



Autoras:

Sofía Herrmann Herrero

Katia Hanis

Curso Académico 2026/2027

Índice

1. Introducción y objetivos de la práctica....3

2. Arquitectura general del sistema....3

2.1. Estados operativos de una estación de riego....4

3. Descripción de los componentes software implementados....5

3.1. Módulo común de comunicaciones (common.py)....5

3.2. Módulo Central (WM_Central.py y dashboard.html)....5

3.3. Monitor de la estación de riego (WM_WS_M.py)....6

3.4. Motor y actuador de la estación (WM_WS_E.py)....6

3.5. Aplicación del operario de campo (WM_FO.py)....7

4. Protocolos de comunicación y persistencia de datos....7

4.1. Protocolo de tramas sobre Sockets TCP (<STX><DATA><ETX><LRC>)....7

4.2. Arquitectura de eventos en Apache Kafka....8

4.3. Base de datos relacional SQLite (central.db)....9

5. Justificación de las decisiones de diseño y resiliencia....9

6. Guía de despliegue y puesta en marcha....10

6.1. Puesta en marcha del núcleo central (Kafka, SQLite y WM_Central)....10

6.2. Despliegue de estaciones de riego y operarios de campo (docker_run.sh)....11

7. Pruebas realizadas, detección de errores y correcciones aplicadas....11

7.1. Arranque de CENTRAL y comprobación del estado inicial de las estaciones....12

7.2. Registro y alta dinámica de una nueva estación (WS-06)....12

7.3. Prueba de riego en WS-06 sin tener el Engine conectado....13

7.4. Correcciones implementadas en el código y verificación final....14

8. Conclusiones.... 16

1. Introducción y objetivos de la práctica

El presente documento recoge el diseño, la implementación y las pruebas de validación del sistema WaterManagement, desarrollado para la primera entrega de la asignatura de Sistemas Distribuidos. El propósito de este proyecto es simular una plataforma distribuida para la gestión, supervisión y control en tiempo real de la red de riego urbano de una ciudad, abarcando distintos parques y zonas verdes (como River Park Norte, River Park Sur, Riverside Gardens, Central Park Este o North Roundabout).

En un entorno urbano real, el riego manual o sin supervisión centralizada provoca importantes ineficiencias: desde averías o fugas de agua que tardan horas en detectarse hasta la imposibilidad de conocer desde el centro de control qué zonas están regando en cada instante o qué consumo acumulado llevan. Para dar solución a este escenario, WaterManagement plantea una arquitectura distribuida en la que todos sus componentes son programas independientes que se comunican a través de la red sin necesidad de ejecutarse en la misma máquina física. De esta manera, el sistema permite la interacción simultánea de dos tipos de usuarios: el gestor de la red (el ayuntamiento, a través del módulo central) y los operarios de mantenimiento que trabajan sobre el terreno activando y supervisando los riegos mediante su propia aplicación.

Desde el punto de vista técnico, el objetivo principal de la práctica consiste en integrar y comparar dos paradigmas de comunicación fundamentales en los sistemas distribuidos actuales:

- Comunicación directa mediante Sockets TCP: empleada para los enlaces de control, registro, autenticación y comprobación periódica del estado de salud de las estaciones, definiendo sobre el flujo de transporte un protocolo propio de empaquetado de tramas a nivel de aplicación con detección de errores.
- Comunicación asíncrona orientada a eventos mediante Apache Kafka: utilizada para desacoplar las solicitudes de los operarios, el envío de órdenes hacia los actuadores de las estaciones y la transmisión continua de telemetría de caudal y volumen en tiempo real.

Adicionalmente, el sistema incorpora persistencia relacional mediante una base de datos SQLite gestionada por el nodo central y se empaqueta íntegramente en contenedores Docker para garantizar su portabilidad y permitir su despliegue distribuido entre un entorno Cloud (como Railway o AWS) y los ordenadores del laboratorio.

2. Arquitectura general del sistema

La arquitectura de WaterManagement se organiza en tres grandes subsistemas que colaboran a través de la red, siguiendo el modelo conceptual establecido en la especificación de la práctica:

- Subsistema Central (Core System): está formado por el proceso principal WM_Central y la

base de datos relacional SQLite (central.db). Actúa como el punto de coordinación y gobierno de toda la red: autentica las estaciones cuando se conectan, valida las peticiones de los operarios, autoriza o deniega los servicios de riego, registra el histórico de consumos y ofrece tanto un panel de control por consola como una interfaz web en tiempo real.

• Estaciones de Riego (Watering Stations, WS): cada estación instalada en un parque o jardín cuenta con un identificador único (por ejemplo, WS-01) y simula tres dispositivos físicos: un caudalímetro (que mide el caudal instantáneo suministrado en litros por minuto), un detector de fugas (que alerta ante pérdidas anómalas o fallos en la instalación) y el actuador de la electroválvula (que abre o cierra el paso del agua). A nivel de software, cada estación se descompone en dos procesos independientes comunicados entre sí por sockets: el Monitor (WM_WS_M), encargado de vigilar la salud del hardware y mantener el enlace con la central, y el Motor o Engine (WM_WS_E), encargado de accionar la electroválvula y calcular el consumo.

• Operarios de Campo (Field Operators, FO): representados por la aplicación cliente WM_FO, que permite al personal de mantenimiento consultar el estado de las estaciones disponibles en la ciudad, solicitar la activación del riego en una estación concreta durante un tiempo determinado y hacer el seguimiento en directo del volumen suministrado hasta la finalización del servicio.

2.1. Estados operativos de una estación de riego

Para reflejar con exactitud la situación de cada punto de la red, una estación de riego puede encontrarse en cualquiera de los cinco estados siguientes, los cuales se representan tanto en la consola de la central como en el mapa interactivo del panel web mediante un código de colores específico:

• Disponible (color verde): la estación está conectada a CENTRAL, tanto su proceso Monitor como su proceso Engine funcionan sin incidencias y se encuentra en reposo, lista para atender una solicitud de riego.

• Regando (color verde parpadeante): la estación funciona correctamente y tiene su electroválvula abierta suministrando agua en ese instante. Mientras permanece en este estado, CENTRAL muestra en tiempo real el caudal instantáneo en litros/minuto, el volumen acumulado en litros y el identificador del operario que solicitó el servicio.

• Fuera de servicio (color naranja): la estación no sufre ninguna avería física, pero ha sido bloqueada deliberadamente por orden directa de CENTRAL. Si estaba regando en el momento del bloqueo, cierra su electroválvula de inmediato y rechaza cualquier nueva petición hasta que CENTRAL vuelva a activarla.

• Fuga o avería (color rojo): el Monitor de la estación mantiene la conexión con CENTRAL, pero ha detectado un fallo en el sensor de fuga o ha perdido la comunicación con el proceso

Engine. Por seguridad, si había un riego en curso se interrumpe al instante y la estación queda inhabilitada hasta que la incidencia se resuelve.

• Desconectada (color gris): CENTRAL no tiene conexión activa con el Monitor de la estación (ya sea porque aún no se ha iniciado o porque se ha caído el proceso o la red). En esta situación CENTRAL desconoce el estado real de la estación y no permite operarla.

3. Descripción de los componentes software implementados

Toda la solución se ha desarrollado en lenguaje Python (versión 3.12), estructurando el código en cinco ficheros principales junto con la plantilla de la interfaz web y los ficheros de configuración para su despliegue en Docker.

3.1. Módulo común de comunicaciones (common.py)

Con el fin de modularizar el código y evitar duplicidades entre los cuatro ejecutables del sistema, hemos creado el módulo common.py. Este archivo reúne la implementación a bajo nivel del protocolo de sockets (las funciones de saludo inicial client_handshake y server_handshake, el cálculo del byte de redundancia longitudinal lrc mediante la operación XOR byte a byte, y las funciones send_frame y recv_frame para enviar y recibir tramas verificadas con confirmación ACK/NACK). Asimismo, define las constantes de estado, las secuencias de escape ANSI para colorear la salida en terminal, los nombres de los cinco tópicos de Apache Kafka y las funciones auxiliares para instanciar productores y consumidores JSON con reintentos de conexión.

3.2. Módulo Central (WM_Central.py y dashboard.html)

El archivo WM_Central.py implementa la lógica de gobierno de toda la red mediante una arquitectura concurrente basada en múltiples hilos de ejecución sincronizados con un cerrojo reentrante (threading.RLock) para proteger el diccionario de estado de las estaciones y los accesos a la base de datos:

• Servidor concurrente de Sockets TCP: escucha en el puerto indicado por línea de comandos (por defecto 5000) y lanza un hilo independiente (_handle_station_socket) por cada Monitor que establece conexión. En este hilo procesa las tramas de alta y autenticación (REGISTRO), actualiza la marca de actividad ante cada latido (PING) con un tiempo máximo de espera de 15 segundos, y gestiona las alertas de avería (LEAK) y recuperación (FIXED). Si el socket se cierra o vence el timeout sin recibir latidos, el bloque finally limpia la sesión, notifica al operario si había un riego activo y pasa la estación automáticamente a estado Desconectada.

• Consumidores y publicadores de Apache Kafka: cuenta con tres hilos dedicados a Kafka. El primero escucha en el tópico fo_requests las solicitudes de riego de los operarios; el segundo escucha en ws_events las confirmaciones de apertura de válvula, la telemetría cada segundo y

los avisos de fin de riego procedentes de los motores; y el tercero publica cada 2 segundos en el tópico ws_status una instantánea completa del estado de la red.

• Consola interactiva y Panel Web (dashboard.html): desde la propia terminal de WM_Central, el administrador dispone de comandos para imprimir la tabla de estado con colores (p), ordenar un riego manual en una estación o en todas a la vez (s <WS|ALL> [segundos]), bloquear estaciones poniéndolas fuera de servicio (b <WS|ALL>), reactivarlas (a <WS|ALL>) y consultar los últimos riegos archivados en SQLite (h). En paralelo, levanta un servidor HTTP multihilo en el puerto 8080 que sirve la aplicación web dashboard.html, ofreciendo en tiempo real un mapa vectorial de los parques de la ciudad, la tabla de monitorización, el estado de los sensores de cada estación y la consola de eventos.

3.3. Monitor de la estación de riego (WM_WS_M.py)

El proceso WM_WS_M.py actúa como vigilante local de la estación y puente de control hacia la central. Recibe como parámetros el puerto local en el que escuchará a su Engine, la dirección IP y puerto de WM_Central y el identificador único de la estación (junto con su ubicación opcional mediante el argumento --loc). En su bucle principal realiza dos tareas de forma continua:

• Enlace con WM_Central: mediante el método ensure_central_connection, establece conexión TCP con el servidor central, ejecuta el handshake inicial y envía la trama REGISTRO#<ID>#<UBICACION>. Si la central se encuentra caída o se reinicia en mitad de la ejecución, el Monitor captura la excepción de red y reintenta la reconexión automáticamente en cada ciclo, enviando además un latido PING#<ID> cada 3 segundos mientras la conexión permanece activa.

• Supervisión del Engine: abre un socket servidor local en espera de que se conecte el proceso WM_WS_E. En cuanto el Engine se conecta, el Monitor le transfiere dinámicamente su identidad enviándole la trama ASSIGN#<ID>. A partir de ese momento, cada segundo le envía la trama de comprobación HEALTH. Si el Engine responde OK, el Monitor notifica FIXED#<ID> a la central para que la estación quede Disponible; si el Engine no se ha conectado aún, cierra la conexión o responde KO, el Monitor envía de inmediato LEAK#<ID> a la central para declararla en estado de Fuga.

3.4. Motor y actuador de la estación (WM_WS_E.py)

El proceso WM_WS_E.py simula los elementos físicos de riego. Al iniciarse únicamente conoce la dirección del broker Kafka y la dirección del socket de su Monitor, por lo que no tiene ningún identificador de estación fijado en código. Su primer paso consiste en conectarse por socket al Monitor y esperar en un evento de sincronización (assigned_event) hasta recibir la trama ASSIGN#<ID>. Una vez conoce su identidad, lanza el hilo consumidor de Kafka sobre el tópico ws_commands filtrando las órdenes dirigidas a su estación:

• Ciclo de riego y telemetría: al recibir una orden START (o al activarse manualmente en la estación), comprueba que su estado sea disponible y que no exista avería activa, abre la electroválvula simulada, publica el evento STARTED en el tópico ws_events e inicia un hilo (_water_loop) que cada segundo genera un caudal aleatorio entre 8,5 y 11,5 L/min, incrementa el volumen acumulado (sumando caudal/60 litros por segundo) y envía un mensaje TELEMETRY a CENTRAL. Al alcanzar la duración programada, cierra la electroválvula y emite el evento FINISHED con el volumen total y el tiempo empleado.

• Simulación interactiva de averías y control local: la consola del Engine ofrece un menú en tiempo real donde el usuario puede pulsar la tecla k para conmutar el estado del sensor de fugas entre normal (OK) y averiado (KO). Si se pulsa k mientras la estación está regando, el propio Engine cierra la electroválvula inmediatamente con motivo LEAK y pasa a responder KO al sondeo HEALTH del Monitor. Asimismo, permite iniciar un riego manual pulsando s y detenerlo pulsando f.

3.5. Aplicación del operario de campo (WM_FO.py)

El programa WM_FO.py proporciona la interfaz de trabajo para los jardineros y operarios de mantenimiento. Al arrancar recibe la dirección del broker Kafka y el identificador del operario (por ejemplo, FO-01). Lanza un hilo consumidor en segundo plano suscrito a los tópicos ws_status (para mantener actualizada la lista de estaciones de la red) y fo_replies (filtrando exclusivamente los mensajes cuyo campo op coincide con el identificador del operario).

En modo interactivo permite listar el estado y caudal de todas las estaciones (comando l) y solicitar un riego (comando r <WS> <segundos>), mostrando por pantalla cada paso de validación recibido desde CENTRAL, la telemetría en directo segundo a segundo y el cuadro resumen final al concluir el servicio. Además, implementa el modo de ejecución por lotes mediante el parámetro --file operaciones.txt: lee secuencialmente las líneas del fichero (con formato WS-ID, segundos), envía la primera solicitud y bloquea el avance mediante un evento (service_done_event) hasta recibir el mensaje SUMMARY o DENY de CENTRAL. Una vez concluido cada servicio, espera estrictamente los 4 segundos exigidos en el enunciado antes de enviar la siguiente petición.

4. Protocolos de comunicación y persistencia de datos

4.1. Protocolo de tramas sobre Sockets TCP (<STX><DATA><ETX><LRC>)

Dado que el protocolo TCP ofrece un servicio de transporte orientado a flujo de bytes (stream) sin conservar las fronteras entre los mensajes enviados, ha sido necesario definir un protocolo de nivel de aplicación entre el Monitor y CENTRAL, así como entre el Monitor y el Engine. Siguiendo la recomendación de la especificación, hemos implementado el estándar de empaquetado <STX><DATA><ETX><LRC> con el siguiente funcionamiento:

• Establecimiento de sesión (Handshake): nada más abrir el socket TCP, el cliente transmite un byte de consulta ENQ (0x05) y aguarda a que el servidor responda con un byte de confirmación ACK (0x06) antes de enviar datos.

• Estructura de la trama: cada mensaje se codifica en UTF-8 separando la operación y sus parámetros mediante el carácter almohadilla (por ejemplo, REGISTRO#WS-01#River Park Norte). El bloque de datos se precede del byte de control STX (0x02, Start of Text) y se cierra con el byte ETX (0x03, End of Text). A continuación se adjunta un byte LRC (Longitudinal Redundancy Check), calculado como la suma lógica exclusiva (XOR) acumulada byte a byte sobre el contenido de DATA.

• Control de errores y retransmisión: el receptor lee del socket hasta encontrar STX, acumula los bytes hasta ETX y lee el byte siguiente como LRC esperado. Si el LRC calculado localmente coincide con el recibido, devuelve un byte ACK (0x06) y entrega el mensaje a la aplicación; en caso contrario, devuelve un byte NACK (0x15) y el emisor retransmite automáticamente la trama hasta un máximo de 3 intentos.

4.2. Arquitectura de eventos en Apache Kafka

Para la comunicación asíncrona a través de Apache Kafka hemos estructurado el flujo de mensajes JSON en cinco tópicos diferenciados, que WM_Central verifica y crea automáticamente al arrancar mediante KafkaAdminClient:

• Tópico fo_requests: producido por WM_FO y consumido por WM_Central. Transporta las solicitudes de activación de riego indicando el identificador del operario, la estación objetivo, la duración en segundos y la marca de tiempo.

• Tópico fo_replies: producido por WM_Central y consumido por WM_FO. Envía a cada operario los mensajes de progreso (STEP), la autorización de inicio (AUTH), la denegación motivada (DENY), la telemetría en vivo (TELEMETRY) y el informe final del servicio (SUMMARY).

• Tópico ws_commands: producido por WM_Central y consumido por WM_WS_E. Canaliza las órdenes de control hacia los motores de las estaciones (START, STOP, BLOCK y ACTIVATE).

• Tópico ws_events: producido por WM_WS_E y consumido por WM_Central. Permite a los motores notificar la apertura efectiva de la electroválvula (STARTED), las lecturas de caudal y volumen cada segundo (TELEMETRY) y el cierre final del suministro (FINISHED).

• Tópico ws_status: publicado periódicamente por WM_Central y consumido por todas las instancias de WM_FO para conocer el listado global de estaciones y su disponibilidad.

4.3. Base de datos relacional SQLite (central.db)

El módulo WM_Central mantiene la persistencia del sistema en el fichero SQLite central.db a través de tres tablas relacionales: operators (almacena el identificador y nombre de los operarios autorizados, precargando de FO-01 a FO-05), stations (registra el identificador, ubicación y estado de cada estación, precargando de WS-01 a WS-05, e insertando dinámicamente mediante una cláusula ON CONFLICT DO UPDATE cualquier nueva estación como WS-06 que se dé de alta en caliente) y watering_history (tabla de auditoría donde se inserta cada servicio finalizado con su marca temporal de inicio y fin, operario, estación, volumen total en litros, duración en segundos y causa de cierre).

5. Justificación de las decisiones de diseño y resiliencia

A lo largo del desarrollo y depuración del sistema hemos adoptado una serie de decisiones de diseño encaminadas a cumplir todos los requisitos funcionales y de tolerancia a fallos de la práctica:

• 1. Elección híbrida de Sockets TCP y Apache Kafka: Utilizar Sockets TCP entre el Monitor y CENTRAL (y entre el Monitor y el Engine) resulta idóneo porque la supervisión de salud requiere una sesión punto a punto continua donde la caída de un extremo o el corte de la conexión pueda detectarse en pocos segundos (mediante los sondeos HEALTH cada segundo y los latidos PING cada 3 segundos). Por el contrario, emplear Apache Kafka para comunicar a los operarios con CENTRAL y con los motores permite desacoplar espacial y temporalmente a los participantes: CENTRAL no necesita conocer las direcciones IP de los operarios ni abrir sockets hacia los motores, pueden incorporarse nuevas instancias de operarios o estaciones en cualquier momento, y el flujo continuo de telemetría por segundo se distribuye sin bloquear los hilos de control.

• 2. Desacoplamiento interno de la estación en Monitor y Engine: Separar cada estación en dos procesos independientes (WM_WS_M y WM_WS_E) garantiza que un fallo o cierre abrupto del motor (por ejemplo, al detener el proceso con Ctrl+C) no deje a la estación sin capacidad de reportar la incidencia. Aunque el Engine caiga, el Monitor sigue en ejecución, detecta en el siguiente segundo que el socket local no responde al mensaje HEALTH y transmite de inmediato la trama LEAK#<ID> a CENTRAL para abortar el servicio y marcar la estación en rojo. Además, al hacer que el Monitor asigne el ID al Engine por socket (ASSIGN#<ID>), evitamos duplicar identificadores en la configuración.

• 3. Registro inicial en estado de Fuga hasta verificar la conexión del Engine: Cuando el Monitor de una estación se conecta y se autentica en CENTRAL, todavía no existe garantía de que su proceso Engine haya sido arrancado. Si CENTRAL marcase la estación como Disponible nada más conectar el Monitor, un operario podría solicitar un riego sobre una estación que carece de motor para abrir la válvula. Por ello, diseñamos el Monitor con

leak_active = True por defecto y configuramos CENTRAL para que toda estación recién autenticada entre inicialmente en estado FUGA, pasando a DISPONIBLE únicamente cuando el Engine se conecta al Monitor, supera el chequeo HEALTH y el Monitor envía la trama FIXED.

• 4. Confirmación en dos pasos al iniciar el riego (pending_start y timeout de 10 segundos): Cuando CENTRAL valida una petición de riego, no cambia el estado de la estación a REGANDO ni envía la autorización definitiva (AUTH) en el mismo instante de publicar la orden START en Kafka, ya que el Engine podría haber sufrido un fallo justo en ese momento. En su lugar, CENTRAL activa la bandera pending_start = True y lanza un hilo temporizador de 10 segundos (check_watering_timeout). Solo cuando el Engine abre efectivamente la electroválvula y devuelve el evento STARTED por Kafka, CENTRAL cambia el estado a REGANDO y notifica AUTH al operario; si transcurren 10 segundos sin confirmación del Engine, CENTRAL cancela la reserva y notifica la denegación (DENY) al operario.

• 5. Prevención de pérdida de mensajes en Kafka mediante offset earliest y filtrado temporal: En Apache Kafka, cuando un proceso nuevo crea un consumidor con auto_offset_reset='latest', el rebalanceo inicial de particiones puede tardar unos cientos de milisegundos, provocando que se pierdan las primeras respuestas si CENTRAL contesta muy rápido. Para resolverlo sin llegar a procesar mensajes antiguos almacenados en el broker de ejecuciones previas, cada consumidor se inicializa con un identificador de grupo único y auto_offset_reset='earliest', pero filtra internamente los mensajes descartando aquellos cuyo campo timestamp sea anterior al instante de arranque del propio proceso.

• 6. Reinicio seguro de CENTRAL con estado Desconectada: En cumplimiento de la primera regla de negocio del enunciado, cada vez que WM_Central se inicia ejecuta una sentencia UPDATE sobre la tabla stations de SQLite fijando el estado de todas las estaciones conocidas a DESCONECTADA. De este modo, si CENTRAL se reinicia tras una caída, nunca muestra estados antiguos como válidos hasta que los monitores restablecen la conexión por socket.

6. Guía de despliegue y puesta en marcha

Desplegar una aplicación distribuida consiste en poner en funcionamiento cada uno de sus componentes en su entorno de ejecución para que puedan comunicarse a través de la red. Cumpliendo el requisito de la práctica, todos los módulos se despliegan utilizando contenedores Docker sin necesidad de emplear ningún entorno de compilación (IDE) ni modificar el código fuente para parametrizar direcciones IP, puertos o identificadores.

6.1. Puesta en marcha del núcleo central (Kafka, SQLite y WM_Central)

El fichero docker-compose.yml define el contenedor del broker Apache Kafka (en modo KRaft, sin necesidad de ZooKeeper) y el contenedor de WM_Central, montando un volumen persistente

(./data:/data) para conservar la base de datos SQLite central.db. Para iniciarlos y verificar su arranque ejecutamos:

```
docker compose up -d
docker compose logs --tail=30 central
```

Una vez en marcha, el servidor de sockets de CENTRAL queda a la escucha en el puerto 5000, el broker de Kafka en el puerto 9092 y el cuadro de mandos web accesible desde cualquier navegador en http://localhost:8080.

6.2. Despliegue de estaciones de riego y operarios de campo (docker_run.sh)

Para facilitar el lanzamiento de tantas instancias de estaciones (Monitor y Engine) y de operarios como se requiera durante la evaluación, hemos creado el script docker_run.sh, que ejecuta la imagen Docker wm_app pasando todos los argumentos por línea de comandos:

```
# 1. Iniciar el Monitor de una estación (puerto local 6001, CENTRAL en localhost:5000, ID WS-01):
./docker_run.sh monitor 6001 localhost:5000 WS-01 --loc "River Park Norte"

# 2. Iniciar el Engine de la estación (conectado a Kafka en localhost:9092 y al Monitor en localhost:6001):
./docker_run.sh engine localhost:9092 localhost:6001

# 3. Iniciar la aplicación de un Operario en modo interactivo (ID FO-01):
./docker_run.sh fo localhost:9092 FO-01

# 4. Iniciar la aplicación de un Operario en modo automático por lotes (espera 4 s entre servicios):
./docker_run.sh fo localhost:9092 FO-01 --file operaciones.txt
```

En el escenario de despliegue distribuido con la parte central alojada en Cloud (Railway o AWS) y el resto de módulos repartidos en distintos ordenadores del laboratorio, el procedimiento es idéntico: basta con sustituir en los comandos anteriores localhost:5000 y localhost:9092 por la dirección IP pública o el endpoint del proxy TCP proporcionado por la plataforma en la nube.

7. Pruebas realizadas, detección de errores y correcciones aplicadas

Durante el desarrollo hemos llevado a cabo una batería de pruebas sobre el entorno Dockerizado para verificar cada requisito funcional y comprobar cómo se comporta el sistema ante situaciones anómalas. A continuación se documentan las pruebas realizadas, los defectos que nos permitieron detectar en una versión previa y las correcciones aplicadas en la versión final entregada.

7.1. Arranque de CENTRAL y comprobación del estado inicial de las estaciones

En primer lugar, levantamos el entorno con Docker Compose y consultamos los registros de arranque del contenedor central mediante el comando docker compose logs --tail=30 central. Como se observa en la captura, el servidor de sockets se inicia correctamente en el puerto 5000, se enlaza con Kafka, muestra el menú de comandos disponibles y habilita el panel web interactivo en el puerto 8080:

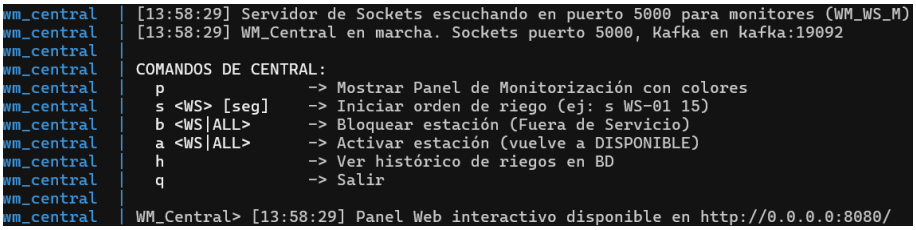


Figura 1: Registros de inicio del contenedor WM_Central.

Seguidamente, abrimos el navegador Chrome en la dirección http://localhost:8080 para inspeccionar el estado inicial de la red. Comprobamos que CENTRAL ha recuperado de la base de datos SQLite las cinco estaciones registradas por defecto (WS-01 a WS-05) y que, al no haberse conectado aún ningún Monitor, todas aparecen correctamente en estado DESCONECTADA (en color gris). Al intentar solicitar un riego sobre WS-01 en este estado, CENTRAL rechaza la operación inmediatamente indicando que la estación está desconectada de la red:

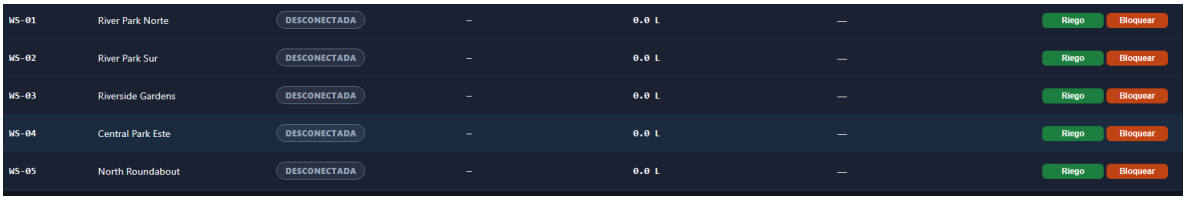


Figura 2: Estado inicial en el panel web tras arrancar CENTRAL con las cinco estaciones desconectadas.

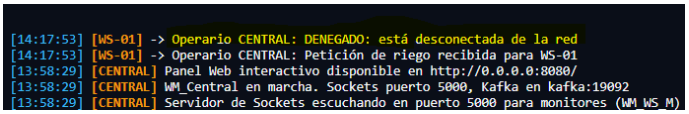


Figura 3: Denegación inmediata de riego en WS-01 al encontrarse desconectada.

7.2. Registro y alta dinámica de una nueva estación (WS-06)

A continuación comprobamos que el sistema permite incorporar y dar de alta en caliente una nueva estación que no existía previamente en la base de datos (WS-06). Para ello ejecutamos en una terminal el contenedor del Monitor asociado a la nueva estación:

```
./docker_run.sh monitor 6006 localhost:5000 WS-06 --loc "Parque Municipal"
```

Este comando inicia el Monitor de WS-06 escuchando en el puerto local 6006 para su Engine y

conectándose al puerto 5000 de CENTRAL para solicitar su registro y autenticación:

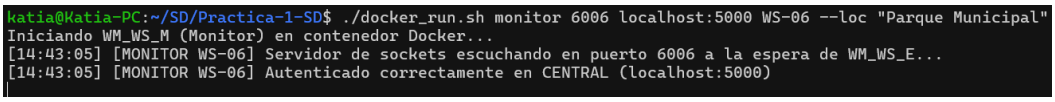


Figura 4: Consola del Monitor de WS-06 autenticándose correctamente en CENTRAL.



Figura 5: Traza en el registro de CENTRAL confirmando el alta de la nueva estación WS-06 (Parque Municipal).

Al realizar esta prueba en nuestra primera versión del código (habiendo arrancado únicamente el Monitor de WS-06, pero sin haber iniciado todavía su proceso Engine), observamos en el panel web que la estación aparecía directamente con el estado DISPONIBLE:



Figura 6: En la prueba inicial, WS-06 pasaba a estado Disponible solo con el Monitor, sin tener el Engine conectado.

7.3. Prueba de riego en WS-06 sin tener el Engine conectado

Para comprobar cómo respondía el sistema ante esa situación, mantuvimos el Monitor de WS-06 en ejecución con su Engine apagado y pulsamos el botón de regar (durante 10 segundos) desde el panel de CENTRAL, analizando tanto la interfaz web como los mensajes de consola de CENTRAL y del Monitor:



Figura 7: Comportamiento en la prueba inicial antes de las correcciones: CENTRAL aceptaba el riego en WS-06 sin Engine.

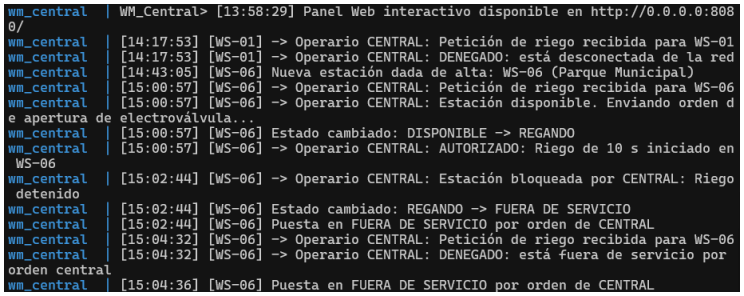


Figura 8: Logs de CENTRAL en la prueba inicial cambiando el estado de Disponible a Regando sin confirmación del Engine.

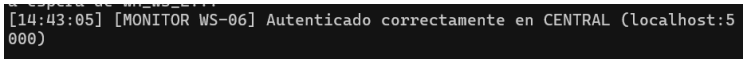


Figura 9: Consola del Monitor de WS-06 esperando la conexión del proceso Engine.

Como muestran las capturas anteriores, en aquella versión preliminar el riego se daba por iniciado erróneamente: CENTRAL aceptaba la solicitud sin que hubiese ningún Engine conectado para abrir la electroválvula, pasando el estado de la estación de Disponible a Regando. Además, aunque el riego se había solicitado por 10 segundos, la estación permanecía en estado Regando indefinidamente con el caudal y el volumen estancados en 0,0 L, por lo que tuvimos que detenerla pulsando el botón Bloquear para forzar su paso al estado Fuera de servicio:



Figura 10: Bloqueo manual de WS-06 desde el panel web, pasando de Regando a Fuera de servicio.

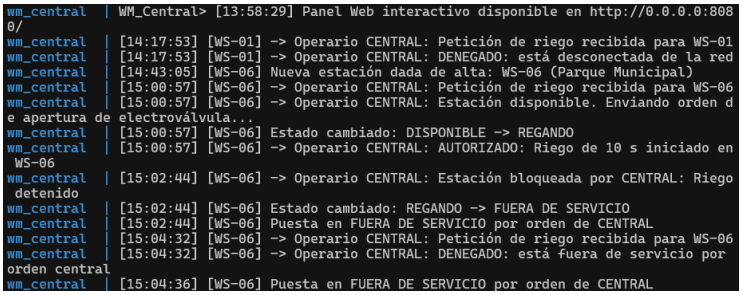


Figura 11: Logs de CENTRAL registrando el paso a Fuera de servicio y denegando peticiones posteriores.

Esta prueba resultó muy útil porque nos permitió identificar cuatro defectos concretos que debían corregirse: en primer lugar, una estación cuyo Engine no está conectado no puede figurar como Disponible; en segundo lugar, CENTRAL no debe autorizar ni dar por iniciado un riego sin recibir confirmación de que el Engine ha comenzado realmente a suministrar agua; en tercer lugar, sin el Engine no se contabilizaba el tiempo ni el volumen de agua; y en cuarto lugar, el indicador visual del estado Regando en el panel web parpadeaba en color azul en lugar del color verde parpadeante especificado en el enunciado.

7.4. Correcciones implementadas en el código y verificación final

Para subsanar de raíz los problemas detectados en la prueba anterior, realizamos tres modificaciones fundamentales en los ficheros WM_WS_M.py, WM_Central.py y dashboard.html:

- 1. Estado inicial en Fuga hasta que el Engine se conecta: en WM_WS_M.py inicializamos el atributo self.leak_active = True y fijamos por defecto health = 'KO' mientras no exista conexión activa con el Engine (self.engine_sock is None). En paralelo, modificamos el manejo del comando REGISTRO en WM_Central.py para que, al conectarse un Monitor, la estación quede registrada inicialmente en estado STATUS_LEAK (Fuga) en lugar de Disponible. De este modo, mientras el Engine está apagado la estación aparece en rojo (Fuga) y rechaza cualquier intento de riego; en el instante en que arrancamos el proceso WM_WS_E

y este responde OK al sondeo HEALTH del Monitor, el Monitor envía la trama FIXED#<ID> a CENTRAL y la estación pasa automáticamente a Disponible (verde). Asimismo, ajustamos el método activate_station de CENTRAL para que solo permita reactivar estaciones que estuvieran en estado Fuera de servicio, impidiendo que se pueda forzar a Disponible una estación con avería activa.

• 2. Confirmación de apertura del Engine con timeout de 10 segundos: en el método request_watering de WM_Central.py eliminamos el cambio inmediato al estado REGANDO. Ahora, al validar una solicitud de riego, CENTRAL marca la estación con pending_start = True, envía la orden START por Kafka y lanza el hilo check_watering_timeout. Únicamente cuando el Engine abre la electroválvula y publica el evento STARTED en el tópico ws_events, CENTRAL cambia el estado a REGANDO y envía el mensaje AUTH al operario. Si transcurren 10 segundos sin recibir el evento STARTED del Engine, el hilo de timeout cancela la solicitud pendiente y envía un mensaje DENY al operario informando de que el Engine no confirmó el inicio del riego.

• 3. Corrección del color del estado Regando en el panel web: en el archivo dashboard.html actualizamos la variable CSS --color-watering y la regla .badge-REGANDO para utilizar el color verde (#22c55e) junto con la animación de parpadeo (blink-anim), cumpliendo fielmente con el formato verde parpadeante exigido en la memoria de la práctica.

Una vez aplicadas estas correcciones, repetimos las pruebas levantando el ecosistema completo (CENTRAL, Monitores, Engines y aplicaciones de Operario) y verificamos satisfactoriamente todos los escenarios de evaluación:

• Ciclo completo de riego con Engine activo: al iniciar tanto el Monitor como el Engine de una estación y solicitar un riego de 10 segundos desde la aplicación del operario (./docker_run.sh fo localhost:9092 FO-01), el Engine confirma la apertura con STARTED, la estación pasa a Regando en verde parpadeante, se actualiza segundo a segundo el caudal (L/min) y el volumen acumulado (L) tanto en CENTRAL como en la consola del operario, y al cumplirse los 10 segundos el Engine cierra la electroválvula, envía el evento FINISHED, CENTRAL almacena el registro en la tabla watering_history de SQLite y la estación regresa automáticamente al estado Disponible.

• Detección de fuga en tiempo real (tecla k) y recuperación: al pulsar la tecla k en la terminal de WM_WS_E durante un riego, el Engine cierra la electroválvula inmediatamente y responde KO al chequeo HEALTH. El Monitor notifica LEAK a CENTRAL en menos de un segundo, el estado cambia a Fuga (rojo) y el operario recibe el aviso de interrupción con el volumen suministrado hasta ese momento. Al pulsar nuevamente k en el Engine, la respuesta vuelve a ser OK, el Monitor envía FIXED y la estación recupera el estado Disponible.

• Ejecución automatizada mediante fichero de lotes (operaciones.txt): al ejecutar la aplicación del operario con el argumento --file operaciones.txt, las solicitudes se procesan de manera

secuencial, esperando a la conclusión de cada riego (o a su denegación si la estación no está disponible) y respetando la pausa exacta de 4 segundos antes de enviar la siguiente orden a CENTRAL.

8. Conclusiones

El desarrollo de la práctica WaterManagement nos ha permitido aplicar sobre un caso práctico completo los principios de diseño y construcción de sistemas distribuidos estudiados en la asignatura. La combinación de Sockets TCP (con empaquetado de tramas <STX><DATA><ETX><LRC> y verificación por XOR) para las labores de registro y control de salud, junto con Apache Kafka para la transmisión asíncrona de órdenes y telemetría, ha demostrado ser una solución muy adecuada: aprovecha la inmediatez de una conexión orientada a flujo allí donde es necesario detectar caídas de nodos, y la flexibilidad del modelo publicador-suscriptor allí donde interesa desacoplar a los clientes del núcleo central.

Asimismo, el proceso de pruebas sobre contenedores Docker puso de manifiesto la importancia de no asumir que todos los procesos de un sistema distribuido están siempre activos al mismo tiempo. La detección de los fallos iniciales cuando el proceso Engine estaba apagado nos llevó a implementar mecanismos de sincronización más rigurosos (como el estado inicial en avería hasta verificar la salud del motor y la confirmación en dos fases con timeout al iniciar cada riego), logrando finalmente un sistema resiliente, coherente en su base de datos SQLite y capaz de recuperarse ante fallos parciales de cualquiera de sus componentes.